

exercise_matching

February 24, 2026

1 Matching Exercise

You have been hired by a mortgage firm (a company that both issues and collects mortgage payments from homeowners). This servicing firm is thinking about entering the market for “no money down” mortgages — mortgages in which the buyer does not have to pay a large percentage of the value of the house when they take out the loan (in the US, most people pay 20% of the value of a house up front when taking out a mortgage). Instead, they just pay a slightly higher interest rate.

Before they start offering this product, though, they want to know the effect this option will have on the likelihood people who take out a mortgage from them will make their payments. In other words, they want to know:

What is the effect of a mortgage being issued as a “no money down” mortgage on the likelihood that the mortgage will enter delinquency (the homeowner will be at least 30 days late on a mortgage payment) during the first two years of the mortgage?

On the one hand, we might expect that someone taking out a “no money down” mortgage would be *less* likely to enter delinquency because they didn’t have to put a lot of cash into a down payment, making it easier for them to afford their payments.

On the other hand, the kind of people who can put 20% down on a house are the kind of people who are able to save effectively, so someone who opts for a “no money down” mortgage with a higher interest rate (instead of saving to pay the 20% down payment) may be more likely to also struggle to make their regular payments.

1.0.1 Data

To answer this question, you have been given [REAL data on a sample of all US Standard single family home mortgages purchased or insured by Freddie Mac](#). This is a random sample of standard mortgages issued in the calendar year 2004.

1.1 Gradescope Autograding

Please follow [all standard guidance](#) for submitting this assignment to the Gradescope autograder, including storing your solutions in a dictionary called `results` and ensuring your notebook runs from the start to completion without any errors.

For this assignment, please name your file `exercise_matching.ipynb` before uploading.

You can check that you have answers for all questions in your `results` dictionary with this code:

```

assert set(results.keys()) == {
    "ex2_share_nmd",
    "ex3_shr_delinquent_nmd",
    "ex3_shr_delinquent_regular",
    "ex6_nmd_regression",
    "ex9_least_balanced",
    "ex11_num_matches",
    "ex11_num_no_money_down",
    "ex12_bivariate_effect",
    "ex13_withcoefficients_effect",
}

```

1.1.1 Submission Limits

Please remember that you are **only allowed THREE submissions to the autograder**. Your last submission (if you submit 3 or fewer times), or your third submission (if you submit more than 3 times) will determine your grade. Submissions that error out will **not** count against this total.

1.2 Data Prep

1.2.1 Exercise 1

To save you some time and let you focus on matching, I've done some pre-cleaning of this data a little. Later on this semester I'll ask you to work with a raw-er and more complete version of this data, but for now, please download the mortgage data here from https://github.com/nickeubank/MIDS_Data/raw/refs/heads/master/mortgages/exercise_matching/standard_mortgages.parquet.

```

[ ]: import pandas as pd

df = pd.read_parquet("standard_mortgages.parquet")
df.head()

```

```

[ ]:
Loan Sequence Number  Original Loan-to-Value (LTV)  Original UPB  \
0          F04Q10000054                      87      127000
1          F04Q10000200                      70      121000
2          F04Q10000236                      80      286000
3          F04Q10000281                      80      114000
4          F04Q10000296                      80      132000

```

```

Original Loan Term  Credit Score  First Payment Date  \
0          360          653      200403
1          360          747      200403
2          180          731      200403
3          180          682      200403
4          360          730      200403

```

```

First Time Homebuyer Flag  Original Interest Rate  \
0              Y              6.050

```

1	N	6.125
2	N	5.125
3	N	5.500
4	N	5.750

	Metropolitan Statistical Area (MSA) Or Metropolitan Division	Postal Code \
0	20740.0	54700
1	30700.0	68500
2	NaN	60500
3	NaN	47100
4	NaN	81200

	delinquent
0	0
1	0
2	0
3	0
4	0

1.2.2 Exercise 2

Original Loan-to-Value (LTV) is the share of the original value of the home that was borrowed. A value of 100 means there was no down payment (“no money down”), while higher values indicate a down payment was made.

If you look at the values of **Original Loan-to-Value (LTV)** in the data, you will see there is a distribution of values. *Truly* no money down mortgages are actually pretty rare — about 1.4% of mortgages — but *almost* no money down mortgages (mortgages where the borrower only pays a 5% down payment) are a lot more common (about 10% of mortgages). So let’s focus our analysis on those 95% LTV mortgages — they aren’t *quite* no money down, but they’re pretty close. From this point forward, when I say “no-money-down mortgages,” I mean 95% LTV mortgages.

To keep things simple, let’s compare these (almost) no money down mortgages to mortgages with a 20% down payment (LTV value of 80). This is the modal mortgage in the US.

Once you’ve limited your data to mortgages with LTVs of 95% and 80%, what share of the mortgages you still have were no-money-down (LTV 95%) mortgages? Store your answer under the key “ex2_share_nmd”.

```
[ ]: ltv= "Original Loan-to-Value (LTV)"
df_ltv9580 = df[df[ltv].isin([95, 80])]
ex2_share_nmd = (df_ltv9580[ltv] == 95).mean()

results = {}
results["ex2_share_nmd"] = ex2_share_nmd

results["ex2_share_nmd"]
```

```
[ ]: 0.19942642841385397
```

1.2.3 Exercise 3

Now let's see how delinquency rates compare between regular and no-money-down mortgages. Store the share of mortgages that are ever delinquent under the keys "ex3_shr_delinquent_regular" and "ex3_shr_delinquent_nmd"

```
[ ]: df_regular = df_ltv9580[df_ltv9580[ltv] == 80]
      ex3_shr_delinquent_regular = df_regular["delinquent"].mean()

      df_nmd = df_ltv9580[df_ltv9580[ltv] == 95]
      ex3_shr_delinquent_nmd = df_nmd["delinquent"].mean()

      results["ex3_shr_delinquent_regular"] = ex3_shr_delinquent_regular
      results["ex3_shr_delinquent_nmd"] = ex3_shr_delinquent_nmd

      results["ex3_shr_delinquent_regular"], results["ex3_shr_delinquent_nmd"]
```

```
[ ]: (0.06337834114081015, 0.1338495575221239)
```

1.2.4 Exercise 4

Now, is this difference the result of not having to put money down, or is it the result of different lending standards?

Let's take a look at a few descriptive statistics to try and answer this question.

First, plot two histograms on the same plot — one with the credit score for regular mortgage borrowers, and one with the credit score for no-money-down mortgage borrowers. Then do the same with the size of the mortgage ("Original UPB") and original interest rates ("Original Interest Rate").

Make sure you're plotting densities not frequencies.

Do these look like the same borrowers? (High credit scores mean people are deemed to be at lower risk of defaulting on their loans).

```
[ ]: import matplotlib.pyplot as plt

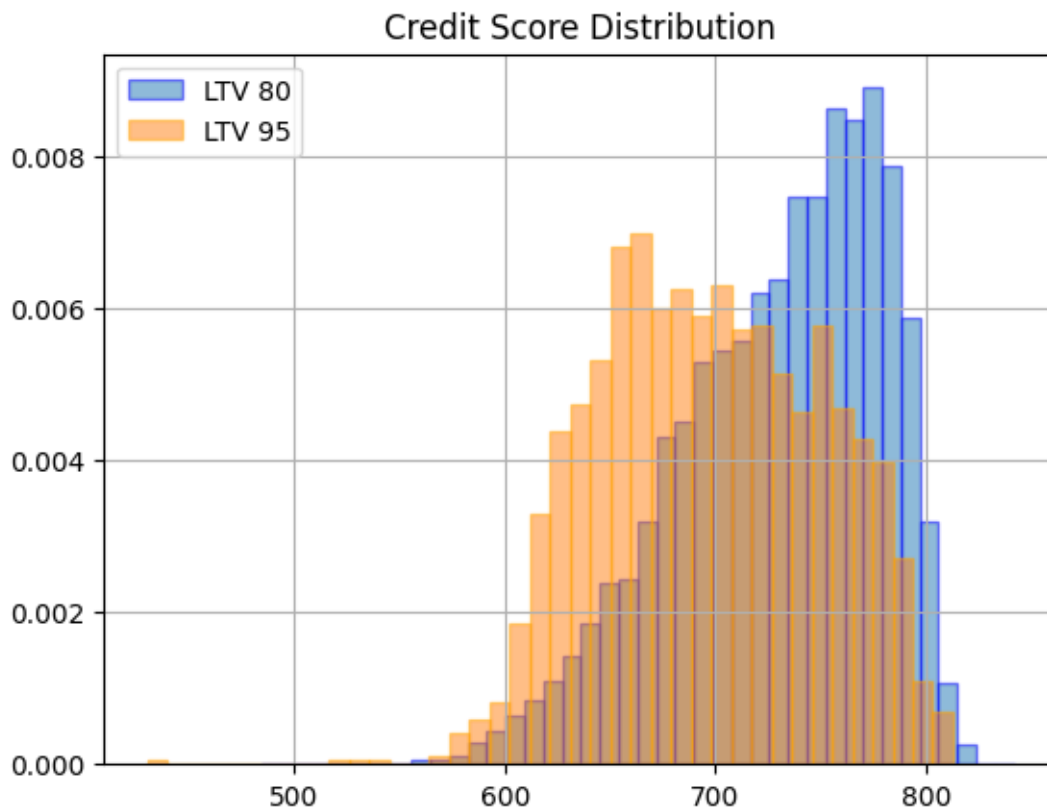
      plt.figure()
      plt.hist(df_regular["Credit Score"], bins=40, density=True, alpha=0.5,
               label="LTV 80", edgecolor="blue")
      plt.hist(df_nmd["Credit Score"], bins=40, density=True, alpha=0.5, label="LTV
               95", edgecolor="orange")
      plt.grid()
      plt.legend()
      plt.title("Credit Score Distribution")
      plt.show()
```

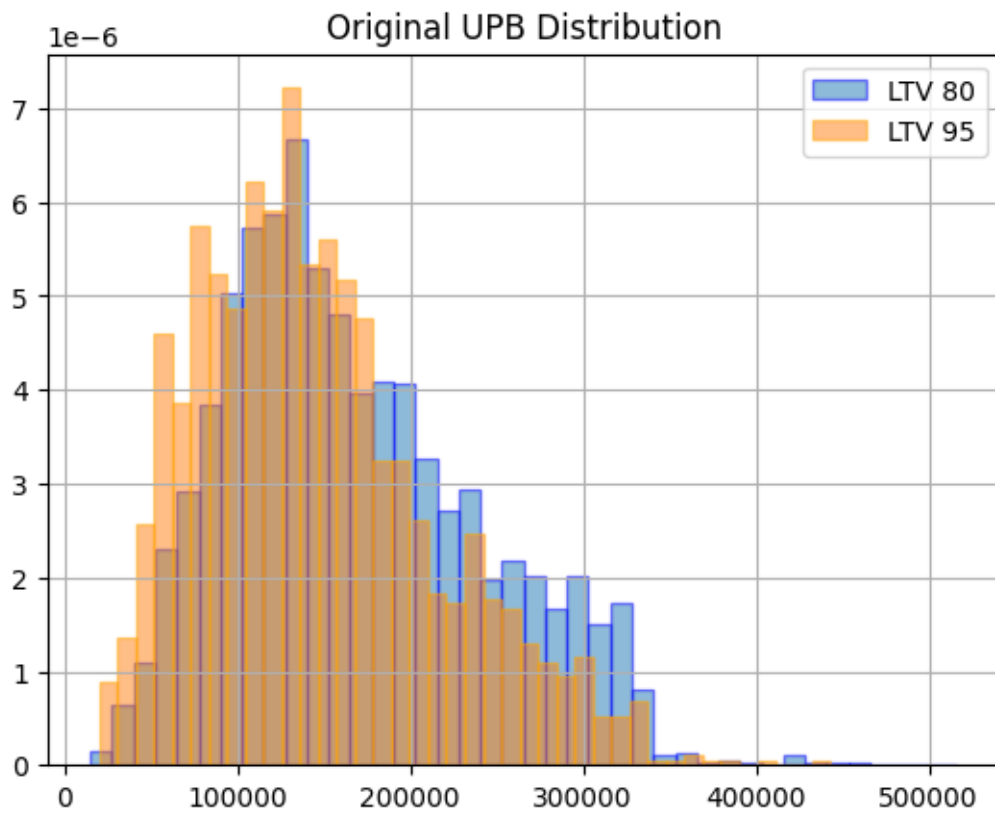
```

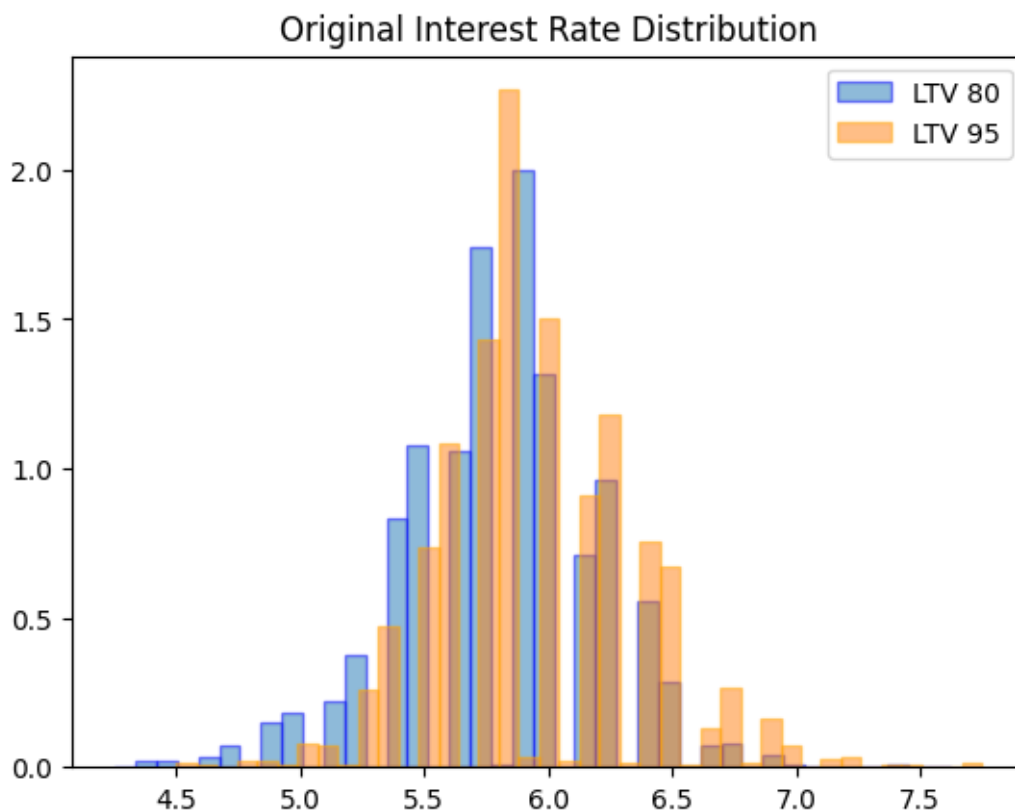
plt.figure()
plt.hist(df_regular["Original UPB"], bins=40, density=True, alpha=0.5,
        label="LTV 80", edgecolor="blue")
plt.hist(df_nmd["Original UPB"], bins=40, density=True, alpha=0.5, label="LTV
        95", edgecolor = "orange")
plt.legend()
plt.grid()
plt.title("Original UPB Distribution")
plt.show()

plt.figure()
plt.hist(df_regular["Original Interest Rate"], bins=40, density=True, alpha=0.
        5, label="LTV 80", edgecolor="blue")
plt.hist(df_nmd["Original Interest Rate"], bins=40, density=True, alpha=0.5,
        label="LTV 95", edgecolor = "orange")
plt.legend()
plt.title("Original Interest Rate Distribution")
plt.show()

```







Answer:

They do not seem like the same type of borrowers. From the graph we can see that borrowers with

1.2.5 Exercise 5

Matching brings value when you have an imbalance in covariates between your control and treatment groups. At the same time, matching only works when there isn't too much imbalance — you need an area of “common support” where you have both control and treatment observations for certain ranges of your covariates.

Looking back at your results from Exercise 6, does it seem like this data might be a good candidate for matching?

Answer:

Yes, this data looks like a good candidate for matching. There is clear imbalance between LTV 80

1.2.6 Exercise 6

To establish a baseline of sorts, regress your “delinquent” variable on:

- whether the loan is a no-money-down mortgage,
- borrower credit score,
- whether borrower is a first time home buyer, and

- the original interest rate.

Drop observations where first-time home buyer is 9 and use this restricted sample moving forward.

Use a linear model and (because the dependent variable is 0-1) heteroskedastic robust standard errors. Store the estimated effect of a mortgage being no-money-down (the coefficient on no-money-down indicator) in `results` under the key `ex6_nmd_regression`.

(Note: if you use the formula api from statsmodels, you may have to change some variable names)

```
[ ]: import statsmodels.api as sm
# No-money-down indicator (LTV 95)
df_ltv9580["nmd"] = (df_ltv9580["ltv"] == 95).astype(float)
#Drop first-time homebuyer == 9
df_ltv9580 = df_ltv9580[df_ltv9580["First Time Homebuyer Flag"] != 9].copy()

#Convert First Time Homebuyer to binary (Y/N → 1/0)
df_ltv9580["first_time"] = (df_ltv9580["First Time Homebuyer Flag"] == "Y").
    ↪astype(float)

#Define X and y
X = df_ltv9580[["nmd", "Credit Score", "first_time", "Original Interest Rate"]]
X = sm.add_constant(X)

y = df_ltv9580["delinquent"]

#Linear probability model with robust SE
model = sm.OLS(y, X).fit()

results["ex6_nmd_regression"] = model.params["nmd"]

results["ex6_nmd_regression"]
```

```
[ ]: 0.03787864553013435
```

1.2.7 Exercise 7

Matching is great because of its ability to reduce dependence on functional form assumptions (do you assume the dependent variable increases linearly with the dependent variable, or the log of the dependent variable, or the square, etc.). This is most powerful where there is some “extrapolation” taking place because of an imbalance in covariates, because there’s no good way to validate those extrapolations. But it also simplifies the general problem of worrying about function forms.

To illustrate, we just fit our model with a range of covariates included as linear controls. But do we know that delinquency decreases linearly with credit scores?

Just looking at delinquency and credit scores, fit a lowess regression (or other model with a flexible functional form) to see if that relationship is indeed linear.

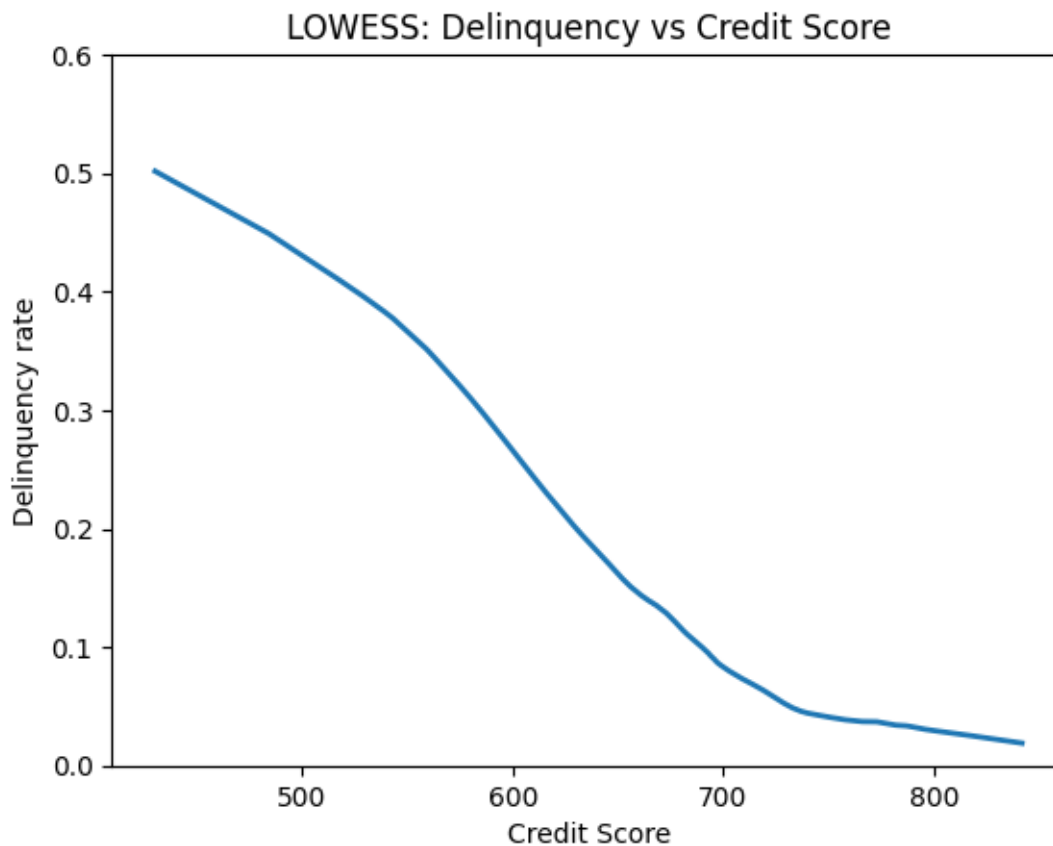
Now do the same for delinquency and the original interest rate. Does that seem linear?

What does this tell you about the accuracy of your regression estimates above?

```
[ ]: import statsmodels.api as sm

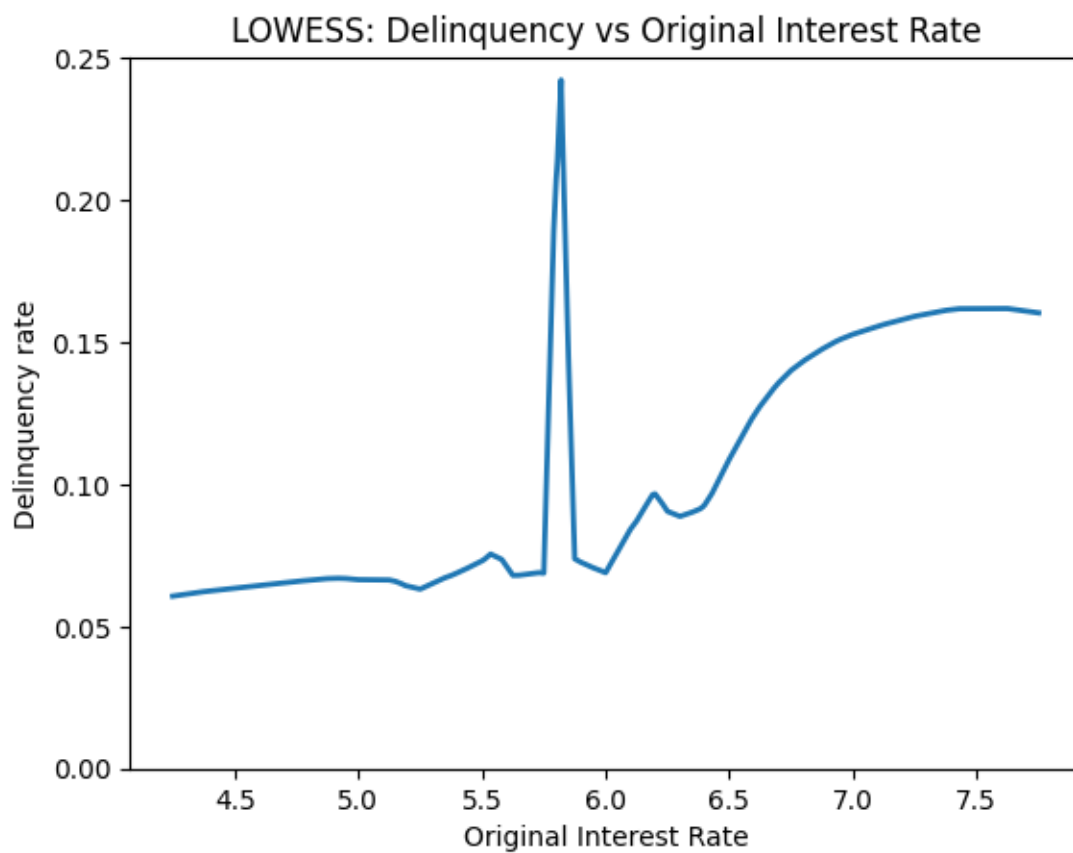
lowess_cs = sm.nonparametric.lowess(
    df_ltv9580["delinquent"],
    df_ltv9580["Credit Score"],
    frac=0.3,
    it=0,          #FIX: no robust outlier reweighting
    return_sorted=True
)

plt.figure()
plt.plot(lowess_cs[:, 0], lowess_cs[:, 1], linewidth=2)
plt.title("LOWESS: Delinquency vs Credit Score")
plt.xlabel("Credit Score")
plt.ylabel("Delinquency rate")
plt.ylim(0, 0.6)
plt.show()
```



```
[ ]: lowess_ir = sm.nonparametric.lowess(
    df_ltv9580["delinquent"],
    df_ltv9580["Original Interest Rate"],
    frac=0.3,
    it=0,
    return_sorted=True
)

plt.figure()
plt.plot(lowess_ir[:, 0], lowess_ir[:, 1], linewidth=2)
plt.title("LOWESS: Delinquency vs Original Interest Rate")
plt.xlabel("Original Interest Rate")
plt.ylabel("Delinquency rate")
plt.ylim(0, 0.25)
plt.show()
```



Answer:

The LOWESS plot of delinquency versus credit score shows a clearly nonlinear relationship. Del.

1.3 Matching!

OK, let's finally do some matching!

To do our matching, we will be using `pymatchit`, a port of the exceptionally good [MatchIt](#) which is only natively available in R. It supports [genetic matching](#), [exact matching](#), [coarsened exact matching](#), [nearest neighbor matching](#), [subclass matching](#), [cardinality matching](#), and more..

This library was just recently created (and I have to imagine with significant LLM help), so it's a little early to be using it. With that said, matching is one area where R packages (namely `MatchIt`) are still well ahead of Python libraries, and after looking things over pretty carefully it seems well implemented — it has a reasonable test suite, includes diagnostic plots (which are something that's so often missing from Python matching and are great for learning), and testing the matching algorithm we're using (coarse exact matching) things seem to match between R and this package. So let's try it. :)

So install it with `pip install pymatchit-causal` (**not** `pip install pymatchit`), and quickly review the [README for the repo here](#).

1.3.1 Exercise 8

Let's do some coarsened exact matching.

In particular, we will match on `credit_score`, `original_interest_rate`, and `first_time_homebuyer_flag`.

Recall that with coarsened exact matching, we have to specify how continuous variables (like `credit_score` and `original_interest_rate`) should be coarsened. This is accomplished with the `cutpoints` keyword. Note the `cutpoints` takes the upper and lower values to be bounds — so `[1, 2, 3]` will create two groups: between 1 and 2 and between 2 and 3. Values below 1 and above 3 will be dropped.

```
from pymatchit import MatchIt

mortgages["first_time"] = (mortgages["first_time_homebuyer_flag"] == "Y").astype("int")

# Deterministic sort to ensure
# random seed generates same result
mortgages = mortgages.sort_values("loan_sequence_number")

m = MatchIt(
    data=mortgages,
    method="cem",
    replace=False,
    random_state=42,
    estimand="ATE",
    cutpoints={
        "credit_score": [500, 575, 650, 675, 700, 720, 740, 760, 780, 800],
        "original_interest_rate": [4.25, 5.25, 6.25, 7.25, 8.25],
    },
)
```

```
# Format: treatment_variable ~ covariate1 + covariate2 + ...
m.fit("no_money_down ~ credit_score + original_interest_rate " "+ first_time")
```

```
[ ]: from pymatchit import MatchIt

mortgages = mortgages.rename(columns={
    "Credit Score": "credit_score",
    "Original Interest Rate": "original_interest_rate",
    "First Time Homebuyer Flag": "first_time_homebuyer_flag",
    "Original Loan-to-Value (LTV)": "ltv",
    "nmd": "no_money_down"
}).copy()

mortgages["first_time"] = (mortgages["first_time_homebuyer_flag"] == "Y").
    ↪astype("int")

# Deterministic sort to ensure
# random seed generates same result
mortgages = mortgages.sort_values("Loan Sequence Number")

m = MatchIt(
    data=mortgages,
    method="cem",
    replace=False,
    random_state=42,
    estimand="ATE",
    cutpoints={
        "credit_score": [500, 575, 650, 675, 700, 720, 740, 760, 780, 800],
        "original_interest_rate": [4.25, 5.25, 6.25, 7.25, 8.25],
    },
)

# Format: treatment_variable ~ covariate1 + covariate2 + ...
m.fit("no_money_down ~ credit_score + original_interest_rate + first_time")
```

Performing cem matching (ATE)...

Matching complete. 8822 observations in matched set.

```
[ ]: <pymatchit.core.MatchIt at 0x14bba3ce0>
```

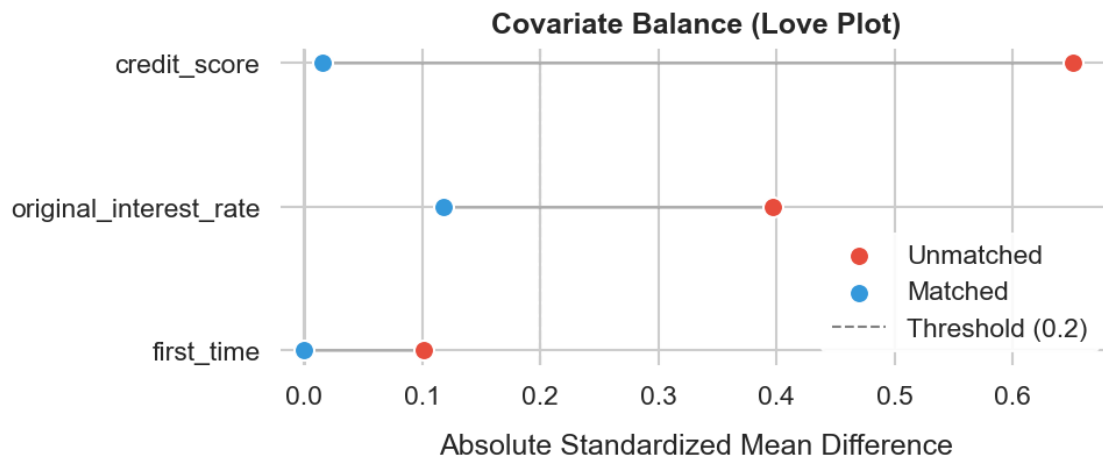
1.3.2 Exercise 9

The first thing to do after matching is to see if the merge worked well.

Use `m.plot(type="balance", threshold=0.2)` to see how balanced the matched coefficients look. Which variable appears not to be that well balanced in the matched data? Store your answer as a

string in "ex9_least_balanced"

```
[ ]: m.plot(type="balance", threshold=0.2)
```



```
[ ]: results["ex9_least_balanced"] = "original_interest_rate"
```

1.3.3 Exercise 10

Let's tighten up our cutpoints to try and get better matches on interest rates. Try:

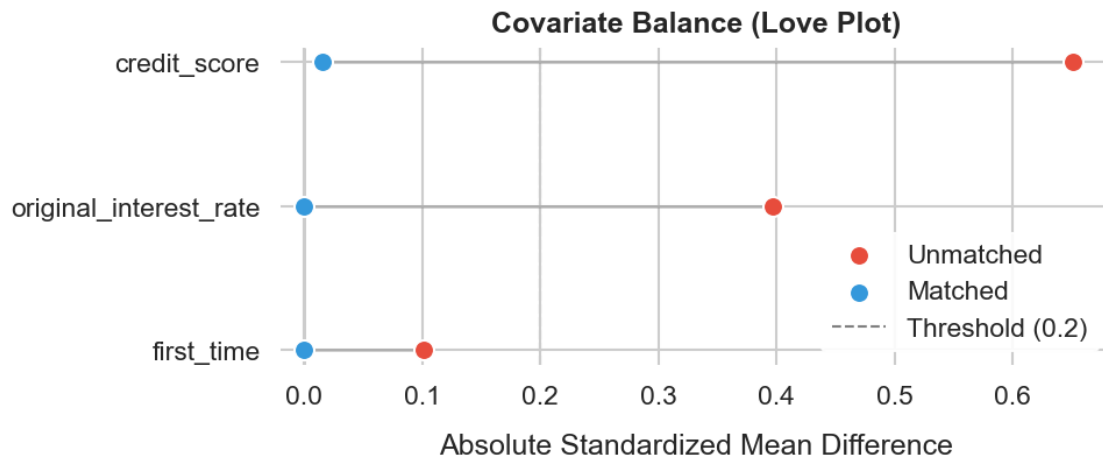
```
cutpoints={
    "credit_score": [500, 575, 650, 675, 700, 720, 740, 760, 780, 800],
    "original_interest_rate": np.arange(4.25, 8.1, 0.1),
}
```

Then check your balance plot.

```
[ ]: m2 = MatchIt(
    data=mortgages,
    method="cem",
    replace=False,
    random_state=42,
    estimand="ATE",
    cutpoints={
        "credit_score": [500, 575, 650, 675, 700, 720, 740, 760, 780, 800],
        "original_interest_rate": np.arange(4.25, 8.1, 0.1),
    },
)

m2.fit("no_money_down ~ credit_score + original_interest_rate + first_time")
m2.plot(type="balance", threshold=0.2)
```

Performing cem matching (ATE)..
 Matching complete. 8433 observations in matched set.



1.3.4 Exercise 11

Given we're still getting a lot of matched observations, so let's tighten credit ratings too! Try:

```
cutpoints = {
    "credit_score": np.arange(500, 800, 20),
    "original_interest_rate": np.arange(4.25, 8.1, 0.1),
}
```

Store your final number of matches in `ex11_num_matches` (you can get this from `len(m.matched_data)`) and the final number of matched treated units (no money down mortgages) in `ex11_num_no_money_down`.

```
[ ]: m3 = MatchIt(
    data=mortgages,
    method="cem",
    replace=False,
    random_state=42,
    estimand="ATE",
    cutpoints = {
        "credit_score": np.arange(500, 800, 20),
        "original_interest_rate": np.arange(4.25, 8.1, 0.1),
    },
)

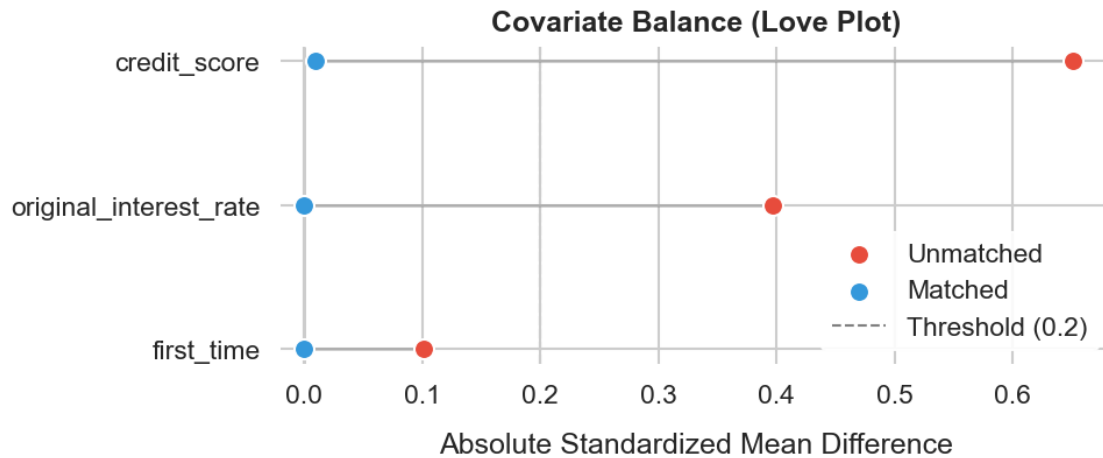
m3.fit("no_money_down ~ credit_score + original_interest_rate + first_time")
m3.plot(type="balance", threshold=0.2)

results["ex11_num_matches"] = len(m3.matched_data)
```

```
results["ex11_num_no_money_down"] = m3.matched_data["no_money_down"].sum()
results["ex11_num_matches"], results["ex11_num_no_money_down"]
```

Performing cem matching (ATE)...

Matching complete. 7486 observations in matched set.



```
[ ]: (7486, 1644)
```

1.4 Analyzing Matches

Assuming you feel ok with the matches at this point, let's do some analyses!

1.4.1 Exercise 12

Using statsmodels, fit a weighted regression using the data in `m.matched_data` with just `no_money_down` as a regressor. Store the resulting coefficient in `"ex12_bivariate_effect"`.

Note that `pymatchit` can provide two different types of weights — weights for estimating ATE, and weights for estimating ATT.

To estimate ATT, the weight of a treated (here, `no_money_down`) observation will always be 1, and the weight for a control observation will be the number of treatment observations in the coarse match group divided by the number of controls.

To estimate ATE, the weights are just the total observations in the coarse match group divided by number of treatment observations (for treated observations) and total observations in the coarse match group divided by the number of control observations (for control observations).

Here, we've specified the estimate is "ATE", so we get weights for estimating ATE.

```
[ ]: #m3.matched_data.columns
X = sm.add_constant(m3.matched_data["no_money_down"])
y = m3.matched_data["delinquent"]
```

```
w = m3.matched_data["weights"]

fit = sm.WLS(y, X, weights=w).fit()

results["ex12_bivariate_effect"] = fit.params["no_money_down"]
results["ex12_bivariate_effect"]
```

```
[ ]: 0.02858644017923669
```

1.4.2 Exercise 13

Because not all our coefficients matched perfectly, its good practice to include the matching variables in your regression in their original form. So estimate your model again, this time with `no_money_down`, `credit_score`, `original_interest_rate` and `first_time` in the regression. Store the new estimated effect in `"ex13_withcoefficients_effect"`.

```
[ ]: X = m3.matched_data[[
    "no_money_down",
    "credit_score",
    "original_interest_rate",
    "first_time"
]]

X = sm.add_constant(X)

y = m3.matched_data["delinquent"]
w = m3.matched_data["weights"]

fit2 = sm.WLS(y, X, weights=w).fit()

results["ex13_withcoefficients_effect"] = fit2.params["no_money_down"]
results["ex13_withcoefficients_effect"]
```

```
[ ]: 0.028086556023927036
```

```
[ ]: results
```

```
[ ]: {'ex2_share_nmd': 0.19942642841385397,
      'ex3_shr_delinquent_regular': 0.06337834114081015,
      'ex3_shr_delinquent_nmd': 0.1338495575221239,
      'ex6_nmd_regression': 0.03787864553013435,
      'ex9_least_balanced': 'original_interest_rate',
      'ex11_num_matches': 7486,
      'ex11_num_no_money_down': 1644,
      'ex12_bivariate_effect': 0.02858644017923669,
      'ex13_withcoefficients_effect': 0.028086556023927036}
```